

Homework 1: Softmax for MNIST Classification

Introduction to Deep Learning

Total: 5 points | Deadline: Oct 11 (Monday noon) 12:00

Overview and learning goals

Build a ten-class linear classifier for handwritten digits using only NumPy for the model and optimization. You will implement a numerically stable softmax function, cross-entropy loss and its gradients, and mini-batch stochastic gradient descent. You will then examine learning curves and compare hyperparameters. The supplied notebook contains the model and experiment scaffold; `mnist_data_loader.py` reads the provided local MNIST files. Complete the notebook's marked TODO cells.

MNIST contains 60,000 training images and 10,000 test images. Each grayscale image is 28×28 pixels, and each label is an integer from 0 to 9. The notebook reserves 10,000 of the original training images for validation. Thus you will train on 50,000 images, choose settings using 10,000 validation images, and evaluate once on the separate 10,000-image test set. The four compressed files are provided in `mnist_data/` beside the notebook and helper module. No download is required when running the notebook; do not submit the data files.

Model and notation

For a mini-batch of B examples, let $X \in \mathbb{R}^{B \times 784}$ contain flattened pixel values scaled to $[0, 1]$, $W \in \mathbb{R}^{784 \times 10}$ be the weight matrix, and $b \in \mathbb{R}^{10}$ be the bias vector. The score and class probability of example i are

$$Z = XW + \mathbf{1}b^\top, \quad P_{ik} = \frac{\exp(Z_{ik} - m_i)}{\sum_{j=0}^9 \exp(Z_{ij} - m_i)}, \quad m_i = \max_j Z_{ij}. \quad (1)$$

Here $y_i \in \{0, \dots, 9\}$ is the true label. The mini-batch objective is

$$L(W, b) = -\frac{1}{B} \sum_{i=1}^B \log P_{i, y_i} + \frac{\lambda}{2} \|W\|_F^2. \quad (2)$$

Regularize W but *not* b . Use the same definition in your code, derivation, and report. The predicted class is $\hat{y}_i = \arg \max_k Z_{ik}$.

Tasks and grading

- Derive the gradients (1 point).** In your report, derive $\nabla_W L$ and $\nabla_b L$ for Eq. (2). Define a one-hot label matrix $Y \in \mathbb{R}^{B \times 10}$ and give the shapes of the resulting gradients. Briefly explain why subtracting m_i leaves the softmax probabilities unchanged but improves numerical stability.
- Implement the classifier (2 points).** Complete the four marked TODO blocks in the notebook: (a) stable softmax probabilities, (b) vectorized loss and gradients, (c) predicted class labels, and (d) mini-batch parameter updates. Use array operations for all examples and classes; loops over epochs and mini-batches are allowed. Do not use loops over individual images or classes in the model implementation. Keep the finite-difference gradient check in the notebook and make it pass.

3. **Train and validate (1 point).** Train the classifier using the provided fixed split and seed. Plot training loss and training accuracy across epochs, and include validation accuracy on the accuracy plot. Report the final training and validation accuracy for each run. The notebook supplies three example configurations; compare at least two learning rates and at least two values of λ while holding the other settings fixed in each comparison.
4. **Evaluate and discuss (1 point).** Select a configuration using validation accuracy, then report its test accuracy *once*. In 4–6 sentences, explain how learning rate and regularization affected your curves or accuracy, whether the training and validation results suggest underfitting or overfitting, and one limitation of a linear pixel classifier.

Rules and submission

- Submit (1) the completed `HW1_Softmax_MNIST.ipynb` with all cells executed, (2) the supplied `mnist_data_loader.py`, and (3) a short PDF report containing your derivation, a compact results table, figures, and discussion. Put your name and student ID in the report.
- Use Python 3, NumPy, Matplotlib, and the Python standard library. Do not use PyTorch, TensorFlow, JAX, scikit-learn classifiers, automatic differentiation, or a prebuilt softmax or cross-entropy loss. The supplied data-loading code may be used as given.
- Use validation data only to compare settings. Do not train on it or use the test set to tune hyperparameters. Keep the random seed and data split in the supplied notebook so results are reproducible.
- Work independently; cite any external material you use. Do not include the MNIST data files in your submission.